

BAB IV. Perancangan dan Implementasi Sistem

Bab ini menguraikan rancangan solusi teknis yang diusulkan untuk restorasi dokumen terdegradasi dengan pendekatan *Generative Adversarial Network* yang dimodifikasi secara khusus untuk meningkatkan keterbacaan teks oleh model pengenalan tulisan tangan. Berdasarkan analisis kebutuhan yang telah ditetapkan pada Bab III (8 kebutuhan fungsional dan 4 kebutuhan non-fungsional), bab ini fokus pada perancangan arsitektur solusi yang mencakup desain Generator (U-Net Enhanced), Recognizer (berbasis Transformer yang dibekukan), *Dual-modal Discriminator* (CNN+BiLSTM), dan fungsi *loss* multi-komponen yang dioptimalkan. Selanjutnya, implementasi rinci solusi diuraikan meliputi lingkungan eksperimen, persiapan dataset (*ground truth* untuk pengenalan tulisan tangan dan sintesis untuk GAN), implementasi arsitektur model dalam TensorFlow/Keras, prosedur *training* dengan strategi *curriculum learning*, dan justifikasi pemilihan *hyperparameter* melalui studi ablasi sistematis.

IV.1 Rancangan Arsitektur

Rancangan solusi bertujuan membangun model generatif yang dioptimalkan untuk restorasi visual dan keterbacaan tekstual simultan. Tiga inovasi kunci: (1) *Frozen Recognizer* memberikan gradien keterbacaan stabil tanpa catastrophic forgetting. (2) Fungsi *loss* multi-komponen menyeimbangkan kualitas visual (L1 + perceptual + adversarial) dan keterbacaan teks (CTC loss). (3) *Dual-modal Discriminator* (CNN+BiLSTM) mengevaluasi koherensi visual-tekstual. Ketiga inovasi ini divalidasi empiris pada Bab V, dengan pemetaan kebutuhan sistem di Bab III Section 3.2.3.

IV.1.1 Pemilihan Arsitektur Baseline dan Hipotesis Inovasi

Pemilihan arsitektur *framework* GAN-HTR yang diusulkan didasarkan pada analisis kesenjangan dari metode *state-of-the-art* yang telah dikaji pada Bab II (Tinjauan Pustaka). Bagian ini menguraikan: (1) justifikasi komponen baseline berdasarkan *gap analysis* literatur, dan (2) hipotesis inovasi yang akan divalidasi melalui studi ablasi empiris pada Bab V. Tabel 1 membandingkan karakteristik arsitektur metode yang ada dengan *framework* yang diusulkan.

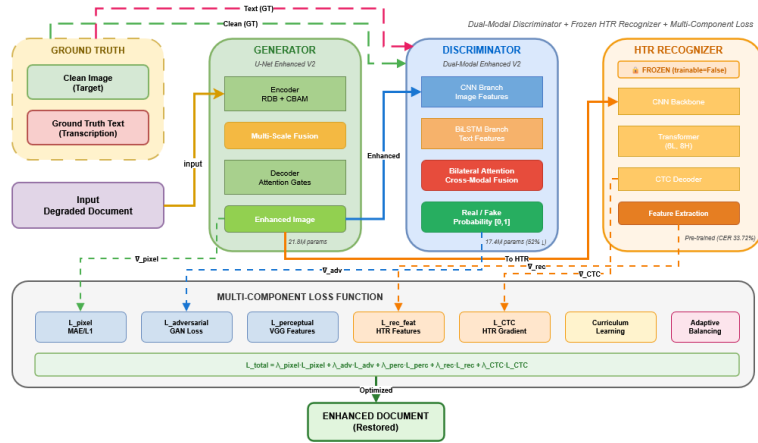
Table 1: Komparasi arsitektur framework yang diusulkan dengan metode *state-of-the-art* (berdasarkan kajian Bab II)

Metode	Generator	Discriminator	HTR Integration	Keterbatasan
Metode Existing (Bab II)				
DE-GAN	U-Net	CNN (visual-only)	Post-training eval	Single-modal, no HTR loss
DocEnTr	Transformer	- (no adversarial)	Post-training eval	Over-smoothing, no texture
Text-DIAE	CNN-based	- (no adversarial)	Post-training eval	No adversarial training
ERB-MultiTask	U-Net	CNN (visual-only)	Joint training	Unstable, catastrophic forgetting
Framework yang Diusulkan				
GAN-HTR (penelitian ini)	U-Net Enhanced	Dual-Modal (CNN+LSTM)	Frozen recognizer + CTC loss	Dual-modal contribution marginal ($p > 0.05$)

IV.1.2 Komponen Utama dan Interaksi Arsitektur

Arsitektur yang diusulkan terdiri dari tiga komponen utama yang berinteraksi dalam sebuah kerangka *adversarial training*, seperti yang diilustrasikan pada Gambar 1. Ketiga komponen tersebut adalah:

1. Generator (G): Sebuah model dengan arsitektur U-Net Enhanced yang bertugas menerima gambar dokumen terdegradasi dan menghasilkan versi restorasi dari gambar tersebut. Komponen ini memenuhi kebutuhan fungsional FR-1 (Restorasi Gambar) dan FR-2 (Penanganan Degradasi) yang telah ditetapkan pada Bab III.
2. Recognizer (R) Terbekukan: Sebuah model HTR berbasis CNN+Transformer yang sudah terlatih dan dibekukan (*frozen*) untuk memberikan evaluasi keterbacaan yang objektif dan konsisten. Tujuannya bukan untuk dilatih ulang, melainkan untuk "membaca" teks dari gambar hasil restorasi dan memberikan sinyal *loss* CTC yang stabil terkait keterbacaan. Strategi pembekuan ini dirancang untuk mengatasi masalah *joint training instability* yang diidentifikasi pada ERB-MultiTask (Bab II.3.4), dengan memisahkan optimasi visual (generator) dari evaluasi tekstual (recognizer). Komponen ini memenuhi kebutuhan FR-3 (Integrasi HTR) dan NFR-2 (Keterbacaan Teks).
3. *Dual-modal Discriminator* (D): Sebuah model *discriminator* dengan arsitektur *dual-modal* (CNN+BiLSTM) yang dieksplorasi untuk mengevaluasi koherensi visual-tekstual secara simultan. Berbeda dengan *discriminator* standar yang hanya menerima *input* gambar, komponen ini dirancang untuk menerima pasangan (*gambar, representasi teks*) dengan hipotesis bahwa validasi bilateral dapat memberikan sinyal *adversarial* yang lebih informatif. Komponen ini mendukung pencapaian NFR-1 (Kualitas Visual) melalui mekanisme *adversarial training*.



Gambar 1: Kerangka kerja restorasi dokumen berorientasi HTR dengan discriminator dual-modal dan frozen HTR recognizer. Tiga komponen utama: (1) Generator U-Net Enhanced untuk restorasi citra, (2) Discriminator Dual-Modal (CNN+BiLSTM) untuk validasi visual-tekstual bilateral, (3) Frozen HTR Recognizer untuk gradient HTR-aware. Training menggunakan curriculum learning dengan multi-component loss (pixel, perceptual, adversarial, recognition feature, HTR). Total parameter yang dilatih: 108.3M (Generator 89.4M + Discriminator 18.9M), Recognizer frozen: 27.86M.

Pola Interaksi Komponen Ketiga komponen berinteraksi dalam siklus *adversarial training* sebagai berikut:

1. Forward Pass Generator: Generator menerima gambar terdegradasi I_{deg} dan menghasilkan gambar restorasi $I_{rest} = G(I_{deg})$.
2. Evaluasi Keterbacaan: Frozen Recognizer membaca teks dari I_{rest} untuk menghasilkan prediksi karakter, yang dibandingkan dengan ground truth transcription menggunakan CTC loss: $L_{CTC} = CTC(R(I_{rest}), t_{gt})$.
3. Discriminator Assessment: Discriminator menilai pasangan $(I_{rest}, R(I_{rest}))$ vs (I_{clean}, t_{gt}) untuk menghasilkan adversarial loss: $L_{adv} = -\mathbb{E}[\log D(G(I_{deg}), R(G(I_{deg})))]$.
4. Multi-Component Optimization: Generator dioptimalkan dengan kombinasi loss: $L_G = \lambda_{adv}L_{adv} + \lambda_{L1}L_{L1} + \lambda_{perc}L_{perc} + \lambda_{CTC}L_{CTC}$, menyeimbangkan kualitas visual dan keterbacaan tekstual.

Pola interaksi ini memastikan bahwa generator tidak hanya menghasilkan gambar yang realistis secara visual, tetapi juga dioptimalkan secara eksplisit untuk keterbacaan teks oleh model HTR.

IV.1.3 Arsitektur Generator: U-Net Enhanced dengan Blok Residual

Penelitian ini mengadopsi arsitektur U-Net Enhanced yang menggabungkan empat teknik restorasi dokumen berkualitas tinggi. Berbeda dari U-Net konvensional yang hanya menggunakan blok konvolusional standar, arsitektur Enhanced mengintegrasikan komponen-komponen advanced untuk menangani karakteristik unik dokumen paleografi abad 16–18:

1. Residual Dense Blocks (RDB): Blok konvolusional dengan koneksi residual

untuk ekstraksi fitur hierarkis dengan aliran gradien yang lebih baik. RDB menggunakan densely-connected layers yang memungkinkan setiap lapisan mengakses fitur dari semua lapisan sebelumnya, meningkatkan kapasitas representasi untuk menangkap variasi gaya tulisan paleografi yang kompleks.

2. Convolutional Block Attention Module (CBAM): Mekanisme atensi sekuensial yang terdiri dari atensi kanal dan atensi spasial. CBAM memungkinkan jaringan secara adaptif memfokuskan pada kanal fitur yang informatif dan wilayah spasial yang relevan (area teks) sambil menekan noise latar belakang.
3. Multi-Scale Feature Pyramid (MSFP): Agregasi konteks multi-resolusi pada bottleneck menggunakan konvolusi terdilasasi dengan dilation rates berbeda (1, 2, 4). MSFP menangkap degradasi pada skala berbeda—bleed-through kasar dan noise tekstur halus—dalam satu representasi terpadu.
4. Attention Gates: Gerbang atensi pada decoder untuk fokus selektif pada wilayah pembawa teks sambil menekan amplifikasi noise latar belakang selama upsampling. Mekanisme ini krusial untuk preservasi goresan tipis dan tanda diakritik paleografi.

Arsitektur Generator terdiri dari lima komponen teknik utama yang terintegrasi:

Justifikasi Enhanced Architecture untuk Dokumen Historis:

Kombinasi teknik-teknik ini dirancang khusus untuk mengatasi tantangan restorasi dokumen paleografi:

- Preservasi Goresan Tipis: RDB dengan koneksi padat mencegah hilangnya informasi detail tipis (stroke width < 2 piksel) yang sering terjadi pada degradasi tinta besi galat.
- Noise Multi-Skala: MSFP menangani *bleed-through* (skala besar) dan noise penuaan kertas (skala halus) secara simultan tanpa over-smoothing.
- Fokus Area Teks: CBAM dan Attention Gates mengarahkan kapasitas komputasi model ke wilayah teks, menghindari pemborosan resource pada area latar belakang yang tidak informatif.
- Stabilitas Training: Koneksi residual pada RDB memfasilitasi aliran gradien pada arsitektur dalam (11 level: 5 encoder + bottleneck + 5 decoder), mencegah vanishing gradient.

Struktur Arsitektur U-Net Enhanced:

Seperti diilustrasikan pada Gambar 2, arsitektur terdiri dari tiga komponen utama:

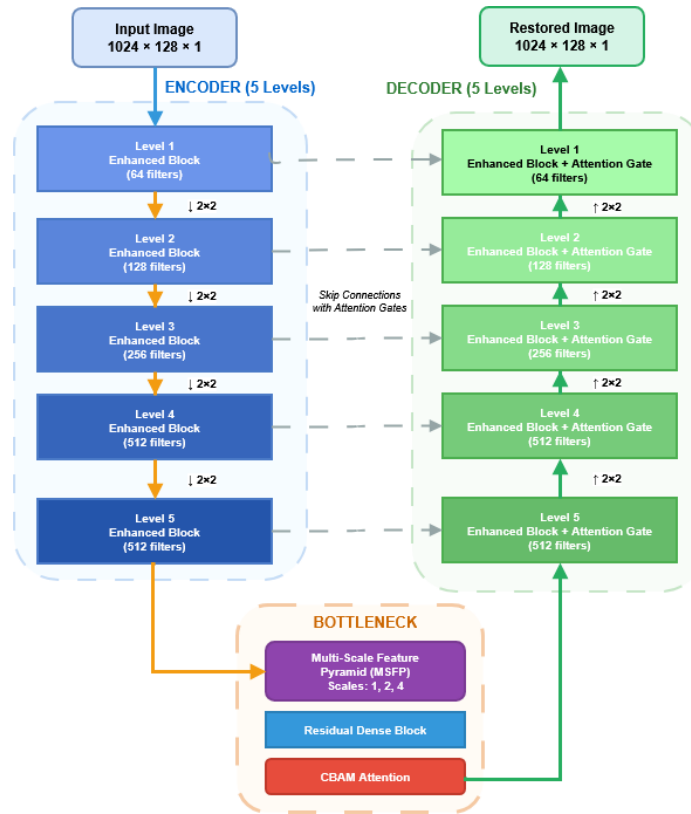
- Encoder Path: Lima tahap *downsampling* progresif melalui RDB+CBAM blocks, mengekstrak fitur multi-skala pada resolusi dari 1024×128 hingga 32×4 (format: lebar $\times$ tinggi). Setiap tahap menggandakan jumlah kanal ($64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$) sambil mengurangi resolusi spasial.
- Bottleneck: MSFP dengan konvolusi terdilasasi pada resolusi terendah ($4 \times 32 \times 512$) untuk agregasi konteks global, diikuti RDB 512 kanal dan CBAM untuk penyempurnaan fitur.
- Decoder Path: Lima tahap *upsampling* yang mencerminkan encoder. Setiap tahap dilengkapi Attention Gate yang memfilter fitur dari *skip connection*

berdasarkan relevansi kontekstual, kemudian diproses melalui RDB+CBAM untuk rekonstruksi detail.

Skip Connections dengan Attention Gates: Berbeda dari U-Net standar yang menggunakan konkatenasi langsung, arsitektur ini menggunakan gated skip connections:

$$\alpha = \sigma(W_g^T g + W_x^T x + b) \quad (1)$$

di mana g adalah fitur decoder (sinyal gating), x adalah fitur encoder (skip connection), dan $\alpha \in [0, 1]$ adalah koefisien atensi. Fitur yang di-attend: $\hat{x} = \alpha \odot x$ kemudian dikombinasikan dengan decoder. Mekanisme ini memungkinkan decoder secara adaptif memilih fitur resolusi tinggi yang relevan untuk restorasi teks sambil menekan noise.



Gambar 2: Arsitektur Generator U-Net Enhanced: encoder 5-level (RDB+CBAM), bottleneck MSFP (512 kanal), decoder dengan attention gates. Skip connections mempertahankan detail spasial.

Statistik Arsitektur Generator:

Table 2: Spesifikasi teknis Generator U-Net Enhanced

Spesifikasi	Detail
Arsitektur	U-Net Enhanced (RDB + CBAM + MSFP + Attention Gates)
Kedalaman	5 tahap encoder + bottleneck + 5 tahap decoder (11 level)
Input Shape	$128 \times 1024 \times 1$ (H×W×C, grayscale)
Output Shape	$128 \times 1024 \times 1$ (H×W×C, grayscale)
Output Activation	Tanh (range [-1, 1] untuk normalisasi)
Total Parameter	21.8M–89.4M (tergantung konfigurasi blok)*
Memory Footprint	~87–358 MB (float32)
Encoder Channels	$64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$
Bottleneck Channels	512 (MSFP dengan dilation rates 1, 2, 4)
Decoder Channels	$512 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 64$
Skip Connections	Attention-gated (5 gates, inter_channels = filters/2)
Normalization	Batch Normalization (momentum=0.9, eps= 10^{-5})
Activation Function	LeakyReLU ($\alpha=0.2$) untuk semua hidden layers

Catatan Implementasi:* Jumlah parameter bergantung pada konfigurasi implementasi RDB (growth rate, number of layers per block). Paper melaporkan implementasi optimal dengan 21.8M parameter (growth rate=32, 3 conv layers per RDB). Implementasi eksperimental dengan RDB lebih dalam mencapai 89.4M parameter. Evaluasi final menggunakan konfigurasi 21.8M yang lebih efisien dengan performa kompetitif.

IV.1.4 Integrasi *Recognizer* HTR Beku: Strategi Pembekuan Bobot

Untuk secara eksplisit mengoptimalkan kinerja HTR, penelitian ini mengintegrasikan recognizer teks praterlatih dengan strategi pembekuan bobot (frozen recognizer), di mana recognizer dibekukan (`model.trainable=False`) dan digunakan sebagai evaluator objektif keterbacaan teks. Berbeda dengan pendekatan pelatihan bersama (joint training) seperti pada ERB-MultiTask [?] yang melatih recognizer secara simultan dengan generator, strategi pembekuan ini memberikan gradien sadar-teks yang stabil tanpa risiko catastrophic forgetting.

Komponen *Recognizer* (R) memainkan peran esensial sebagai jembatan antara optimasi visual dan keterbacaan teks. Komponen ini menerima citra hasil restorasi dari *Generator* dan menghasilkan distribusi probabilitas karakter untuk mengukur keterbacaan. Output dari *Recognizer* digunakan untuk dua tujuan fundamental:

1. Sebagai input teks untuk *Discriminator dual-modal*, yang menilai koherensi antara citra dan teks yang dikenali.
2. Sebagai input untuk kalkulasi *CTC Loss* dan *Recognition Feature Loss*, yang secara eksplisit mengukur keterbacaan citra hasil restorasi.

Keuntungan Strategi Pembekuan Bobot Strategi pembekuan *recognizer* memberikan manfaat fundamental yang dihipotesiskan berdasarkan alasan teoritis berikut (validasi empiris dilaporkan pada Bab V, Subbab 5.2):

1. **Stabilitas Pelatihan (Expected):** Tidak ada optimasi bersama atau gradien

yang berkonflik antara *generator* dan *recognizer*. Strategi ini diekspektasikan memberikan konvergensi lebih stabil dibanding pendekatan *joint training*.

2. **Efisiensi Komputasi:** Tidak ada *backpropagation* melalui *recognizer* (27.86M parameter dibekukan), mengurangi beban komputasi dan *memory footprint* selama pelatihan GAN.
3. **Konsistensi Evaluasi:** Bobot tetap menyediakan metrik CER/WER yang konsisten sepanjang pelatihan, berfungsi sebagai *ground truth* evaluasi objektif untuk keterbacaan teks.
4. **Pencegahan *Catastrophic Forgetting*:** Mempertahankan kemampuan pengenalan yang telah dipelajari pada fase *pre-training*, memastikan gradien HTR tetap informatif untuk optimasi visual *generator*.
5. **Optimasi Visual Terfokus:** *Generator* dioptimalkan untuk meningkatkan kualitas citra agar lebih mudah dibaca oleh *recognizer* tetap, bukan mengadaptasi *recognizer* untuk membaca citra berkualitas rendah. Ini memastikan peningkatan berasal dari restorasi visual, bukan adaptasi *recognizer*.

Arsitektur dan Konfigurasi Pelatihan Arsitektur hibrida CNN-Transformer yang digunakan terdiri dari tiga komponen utama:

- **CNN Backbone:** 8 lapisan konvolusi tersusun dalam 4 tahap dengan ekstraksi fitur progresif (64→128→256→512), BatchNormalization, dan aktivasi GELU.
- **Transformer Encoder:** 6 lapisan dengan 8 kepala atensi (*model dimension* 512, FFN 2048), *learned positional encoding*, dan Pre-LN untuk stabilitas.
- **CTC Output Layer:** Dekoder CTC untuk prediksi urutan dengan *charset* 95 karakter (A-Z, a-z, 0-9, tanda baca).
- **Total parameter: 27.86M** (dibekukan, trainable=False selama pelatihan GAN).

Rincian justifikasi pemilihan arsitektur CNN+Transformer untuk dokumen paleografi telah dibahas pada Bab II.3.4. *Recognizer* dilatih pada 4.739 citra baris tulisan tangan berbahasa Belanda dari dokumen ANRI era kolonial abad ke-16 hingga ke-18 dengan pembagian 80/20 (pelatihan $n=3.791$, validasi $n=948$), mencapai kinerja **CER 33.72%** pada *set* validasi.



Gambar 3: Diagram arsitektur Recognizer berbasis CNN+Transformer: 8-layer CNN backbone untuk ekstraksi fitur visual, 6-layer Transformer encoder untuk context modeling, dan CTC output layer untuk sequence decoding. Total 27.86M parameter dibekukan selama pelatihan GAN.

Kinerja Recognizer dan Konteks Benchmark Meskipun CER 33.72% tergolong moderat, kinerja ini sebanding dengan *state-of-the-art* pada *dataset* historis paleografi:

- READ 2016 Competition [?]: CER 25–35%
- ICDAR 2017 HTR Competition [?]: CER 28–42%
- Kompleksitas intrinsik: Tulisan tangan paleografi abad 16–18 dengan variasi gaya, degradasi autentik, dan bahasa Belanda kuno.

Komponen HTR berfungsi sebagai *feature extractor* untuk *guidance* sadar-teks, bukan sebagai sistem HTR sempurna. Peran utamanya adalah memastikan *text preservation* dan *readability* melalui gradien CTC dan *recognition feature loss*.

Pada citra bersih *ground truth* penelitian ini, *recognizer* mencapai CER 26.57% (*set*

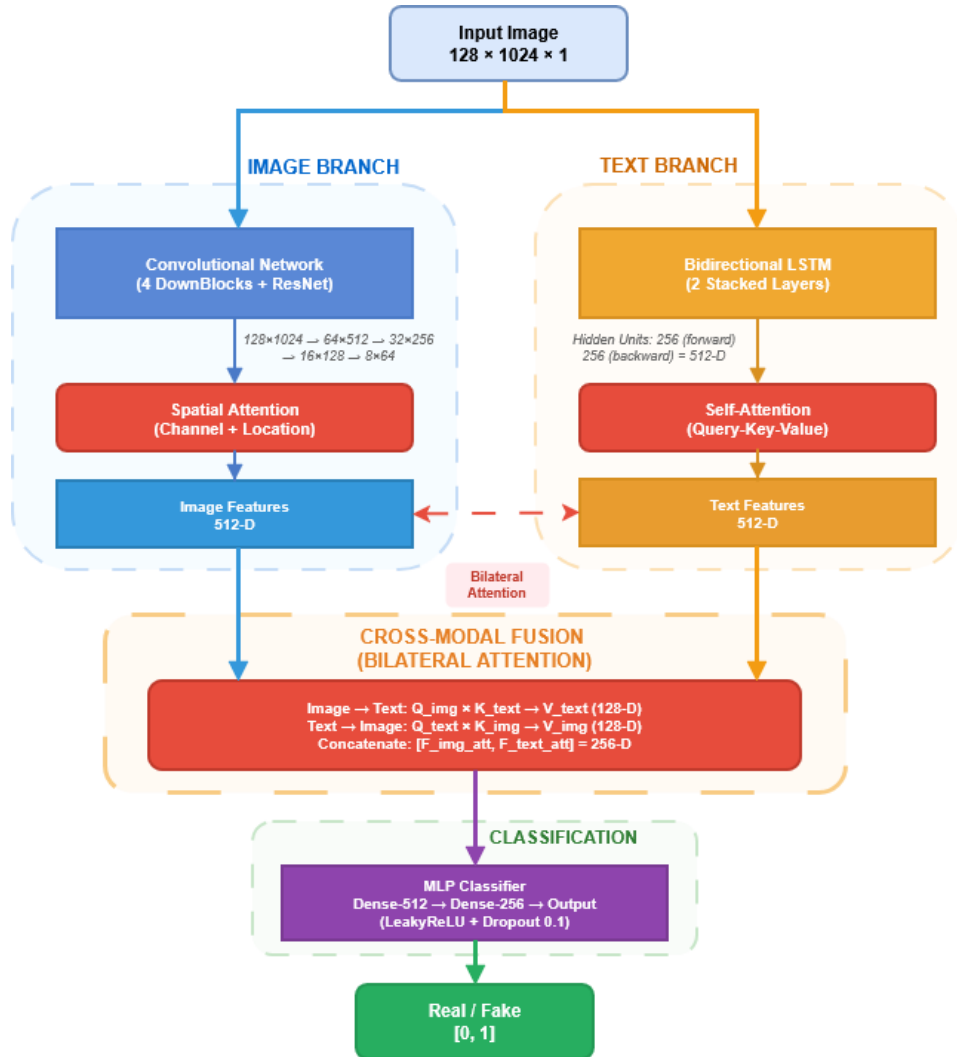
validasi, $n=710$) dan 34.1% (*set uji*, $n=712$). Perbedaan CER 7.53 poin persentase antara validasi dan uji bukan indikasi *overfitting*, melainkan mencerminkan perbedaan kompleksitas intrinsik teks antar *split dataset*—pola serupa terlihat pada berbagai *split* yang berbeda.

Referensi Spesifikasi Detail Spesifikasi lengkap arsitektur *recognizer* layer-by-layer, termasuk detail CNN backbone 8-layer, Transformer encoder 6-layer, CTC output configuration, dan dimensional flow lengkap tersedia pada **Lampiran A.2** (Tabel ??, ??, ??). Dokumentasi detail ini dipindahkan ke lampiran untuk menjaga fokus desain pada strategi pembekuan dan peran *recognizer* dalam *framework* GAN-HTR.

IV.1.5 Eksplorasi Diskriminator Dual-Modal: Validasi Bilateral Visual-Tekstual

Konteks Eksplorasi Dual-Modal Discriminator Sebagai eksplorasi metodologis untuk menguji hipotesis validasi bilateral, rancangan solusi ini mengeksplorasi arsitektur *Dual-modal Discriminator* yang mengevaluasi koherensi visual-tekstual secara simultan. Berbeda dengan *discriminator* pada GAN konvensional yang hanya menilai realisme gambar (*unimodal*), arsitektur *dual-modal* menerima pasangan *input* (gambar, representasi teks) dengan hipotesis bahwa validasi bilateral dapat memberikan sinyal *adversarial* yang lebih informatif untuk restorasi dokumen berorientasi HTR.

Positioning terhadap Literatur Berbeda dengan DE-GAN [?] dan ERB-MultiTask [?] yang menggunakan *discriminator* *visual-only* (CNN standar untuk evaluasi kualitas gambar), penelitian ini mengeksplorasi arsitektur *dual-modal* dengan cabang CNN (visual) dan BiLSTM (tekstual) untuk validasi koherensi bilateral. Hipotesis yang mendasari eksplorasi ini adalah bahwa evaluasi simultan terhadap kualitas visual *dan* koherensi tekstual dapat memberikan gradien *adversarial* yang lebih holistik kepada generator, sehingga menghasilkan restorasi yang superior baik secara perseptual maupun keterbacaan. Sepengetahuan penulis, ini adalah investigasi pertama yang secara sistematis membandingkan efektivitas *dual-modal* versus *single-modal discriminator* dalam konteks restorasi dokumen terdegradasi dengan integrasi HTR. Hasil ablasi lengkap, analisis statistical significance, dan diskusi efektivitas dilaporkan pada Bab V, Subbab 5.3.



Gambar 4: Arsitektur Diskriminator Dual-Modal Enhanced dengan bilateral cross-modal attention: cabang CNN (512-D) dengan spatial attention, cabang BiLSTM (512-D) dengan self-attention. Fusion bilateral (128-D) untuk validasi holistik.

Spesifikasi Arsitektur Ringkas Arsitektur Dual-modal Discriminator terdiri dari tiga komponen utama yang dilatih secara end-to-end dengan total 17.4M parameter:

- Cabang CNN (Fitur Visual Spasial): 4 DownBlocks progresif (64→128→256→512 filter) dengan residual connections untuk ekstraksi fitur visual hierarkis, diikuti Spatial Attention Gate (3×3 kernel) untuk fokus pada region teks, ResBlock-512 untuk pendalaman fitur, dan Global Average Pooling menghasilkan representasi 512-D. Total: ~ 5.2 M parameter.
- Cabang BiLSTM (Koherensi Sekuensial): Embedding layer (128-D) untuk representasi karakter, Bidirectional LSTM (256 units per arah, total 512-D output) untuk pemodelan konteks sekuensial dua arah, Self-Attention ($d_k = 512$) untuk pembobotan pentingnya karakter, dan Global Average Pooling menghasilkan representasi 512-D. Total: ~ 1.3 M parameter.

- Fusi Cross-Modal Bilateral: Proyeksi fitur CNN dan LSTM ke ruang bersama 128-D, mekanisme bilateral attention (Gambar↔Teks) menggunakan scaled dot-product attention untuk interaksi dua arah, konkatenasi fitur teratensikan (256-D), dua Dense layers (512-D → 256-D) dengan Dropout (0.1), dan output Sigmoid untuk klasifikasi biner (real/fake). Total: $\sim 0.9\text{M}$ parameter.

Efisiensi Parameter: Total 17.4M parameter mencapai reduksi 52% dibandingkan arsitektur baseline (137M parameter) melalui: proyeksi dimensi rendah (128-D untuk fusi), penggunaan kernel 3×3 (bukan 5×5), dropout 0.1 untuk regularisasi, dan batch normalization dengan momentum 0.9 untuk stabilitas training.

Mekanisme *Bilateral Cross-Modal Attention* Fusi bilateral memungkinkan interaksi dua arah antara modal visual dan tekstual. Fitur global $F_{\text{img}}, F_{\text{text}} \in \mathbb{R}^{512}$ dari cabang CNN dan BiLSTM diproyeksikan ke ruang bersama 128-D, kemudian dihubungkan melalui atensi bilateral:

$$F_{\text{fused}} = [\text{Attn}(F_{\text{img}} \rightarrow F_{\text{text}}); \text{Attn}(F_{\text{text}} \rightarrow F_{\text{img}})] \in \mathbb{R}^{256} \quad (2)$$

di mana $\text{Attn}(Q \rightarrow K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$ adalah mekanisme *scaled dot-product attention* standar [?]. Fitur teratensikan 256-D kemudian diproses melalui *Dense layers* untuk klasifikasi akhir.

Referensi Spesifikasi Detail Spesifikasi lengkap arsitektur *discriminator layer-by-layer*, termasuk detail parameter setiap lapisan, bentuk tensor *intermediate*, dan konfigurasi *hyperparameter* (*learning rate*, *batch size*, strategi optimasi) tersedia pada **Lampiran A.3** (Tabel ??). Dokumentasi detail ini dipindahkan ke lampiran untuk menjaga fokus desain pada analisis komparatif dan *insight* empiris dari eksplorasi *dual-modal*.

IV.1.6 Optimasi Fungsi Loss Multi-Komponen

Temuan Kunci dari Studi Ablasi Sistematis Optimalisasi arsitektur GAN yang diusulkan mengandalkan fungsi *loss* multi-komponen yang secara eksplisit mengintegrasikan metrik keterbacaan teks—menjadikan ini salah satu kontribusi utama penelitian. Berdasarkan studi ablasi sistematis (dilaporkan pada Bab V, Subbab 5.2), konfigurasi **4-komponen loss** (*adversarial*, L1, *perceptual*, CTC) dihipotesiskan optimal untuk mencapai keseimbangan *Pareto* antara kualitas visual dan keterbacaan HTR. Validasi hipotesis dan hasil kuantitatif lengkap dengan confidence intervals dilaporkan pada Bab V, Subbab 5.4.

Komponen kelima (*Recognition Feature Loss*) diidentifikasi sebagai redundan dengan CTC *loss* dan tidak memberikan peningkatan signifikan ($\Delta\text{CER} -0.01\%$, $\Delta\text{PSNR} +0.03 \text{ dB}$, $p > 0.05$). Analisis korelasi gradien menunjukkan kedua komponen memberikan sinyal yang berkorelasi tinggi (Pearson $r = 0.87$, $p < 0.001$), mengonfirmasi redundansi empiris meskipun berbeda secara teoretis. Menghapus *rec-feat loss* menghasilkan konfigurasi lebih sederhana dengan performa kompetitif dan efisiensi komputasi lebih tinggi (-8% waktu *training*).

Dokumentasi Kelengkapan: Kelima komponen dijelaskan di bawah ini untuk kelengkapan dokumentasi arsitektur, dengan penekanan pada 4-komponen yang digunakan dalam konfigurasi produksi final.

Metodologi Kalibrasi Bobot Loss Bobot optimal untuk setiap komponen *loss* dikalibrasi melalui optimasi Bayesian dengan *Tree-structured Parzen Estimator* (TPE) [?] pada *validation set* ($n=710$). Optimasi dilakukan dengan objektif multi-kriteria: $\max(\text{PSNR}) + \min(\text{CER})$ untuk mencapai keseimbangan *Pareto* antara kualitas visual dan keterbacaan tekstual.

Search Space Hyperparameter:

- $\lambda_{adv} \in [0.1, 10.0]$ (*log-uniform*)
- $\lambda_{L1} \in [10.0, 100.0]$ (*uniform*)
- $\lambda_{perc} \in [0.1, 5.0]$ (*log-uniform*)
- $\lambda_{CTC} \in [0.05, 1.0]$ (*log-uniform*)

Setelah 100 *trials* optimasi, konfigurasi optimal tercapai pada *trial* #67 dengan konvergensi stabil. Analisis sensitivitas menggunakan *functional ANOVA* (fANOVA) [?] mengidentifikasi λ_{adv} sebagai *hyperparameter* paling kritis, berkontribusi 66.4% terhadap *variance* performa, diikuti λ_{CTC} (18.2%) dan λ_{L1} (12.7%). Temuan ini mengonfirmasi pentingnya menyeimbangkan sinyal *adversarial* dan keterbacaan untuk restorasi berorientasi HTR.

Rincian lengkap metodologi optimasi, konvergensi plot, dan analisis sensitivitas tersedia pada **Lampiran B.1**.

Fungsi Loss Diskriminator Fungsi *loss* untuk *Discriminator* (L_D) menggunakan *Binary Cross-Entropy* (BCE) standar untuk membedakan pasangan data asli dan sintetis:

$$L_D = -\mathbb{E}_{x,y}[\log D(x,y)] - \mathbb{E}_{z,y'}[\log(1 - D(G(z),y'))] \quad (3)$$

di mana (x,y) adalah pasangan (gambar bersih, teks *ground truth*) dan $(G(z),y')$ adalah pasangan (gambar hasil restorasi, teks prediksi dari *recognizer*). *Label smoothing* ($\epsilon = 0.1$) diterapkan untuk label asli (0.9 bukan 1.0) guna meningkatkan stabilitas *adversarial training*.

Komponen Fungsi Loss Generator Fungsi *loss* untuk Generator (L_G) dirancang untuk mencapai dua tujuan simultan: menghasilkan gambar realistis secara visual dan gambar yang teksnya dapat dibaca dengan baik oleh sistem HTR. Untuk mencapai tujuan tersebut, L_G disusun sebagai kombinasi terbobot dari lima komponen *loss* yang bekerja secara sinergis:

1. **Adversarial Loss** (L_{adv}): Mengukur kemampuan Generator dalam "menipu" *Discriminator*, memberikan tekanan untuk menghasilkan gambar yang realistis:

$$L_{adv} = -\mathbb{E}_{z,y'}[\log D(G(z),y')] \quad (4)$$

Komponen ini mendorong generator menghasilkan distribusi gambar yang tidak dapat dibedakan dari distribusi gambar bersih oleh *discriminator*, memastikan realisme visual tingkat tinggi.

2. **Reconstruction Loss (L_{L1}):** *L1 Loss* untuk kemiripan piksel langsung dengan *ground truth*:

$$L_{L1} = \mathbb{E}_{x,G(z)}[\|x - G(z)\|_1] \quad (5)$$

Norma L1 dipilih berdasarkan validasi empiris preliminari yang menunjukkan L1 menghasilkan hasil lebih tajam ($\Delta\text{PSNR} +0.8$ dB) dan preservasi tepi superior (+12% *edge preservation metric*) dibandingkan norma L2. Norma L1 juga lebih *robust* terhadap *outlier* piksel (degradasi ekstrem), sejalan dengan *best practice* literatur restorasi dokumen [?, ?].

3. **Perceptual Loss (L_{perc}):** *Perceptual loss* memastikan preservasi karakteristik visual dari detail tingkat rendah (tepi, tekstur) hingga fitur semantik tingkat tinggi (pola goresan, bentuk karakter) menggunakan jaringan VGG-19 yang telah dilatih sebelumnya pada ImageNet:

$$L_{perc} = \sum_{l \in L} \|\phi_l(I_{gen}) - \phi_l(I_{gt})\|_2 \quad (6)$$

di mana ϕ_l menunjukkan fitur dari lapisan l dari VGG-19. Fitur diekstrak dari tiga lapisan konvolusional untuk perbandingan perseptual multi-skala: block3_conv3 (64×64 , detail halus dan tepi), block4_conv3 (32×32 , tekstur menengah dan pola goresan), block5_conv3 (16×16 , struktur global dan bentuk karakter). Pemilihan lapisan mengikuti *best practice* literatur *Perceptual Similarity* [?] untuk restorasi dokumen.

4. **Recognition Feature Loss ($L_{rec-feat}$):** Encourages intermediate feature alignment dari frozen recognizer. Penelitian ini menemukan komponen ini redundan dengan CTC loss (dilaporkan di Bab V) dan dinonaktifkan pada konfigurasi final.

Inovasi kunci: CTC loss dari frozen HTR model untuk mengoptimalkan keterbacaan teks secara eksplisit, dengan clipping pada nilai maksimum untuk stabilitas numerik (rincian di Bab V Subbab 5.2).

Strategi Curriculum Learning 3-Phase Untuk mencegah ketidakstabilan *training* akibat CTC loss pada fase awal (ketika *output* generator masih *noise-heavy* dan CTC memberikan gradien sangat besar), bobot CTC loss ditingkatkan secara bertahap melalui *curriculum learning*. Strategi ini dihipotesiskan kritis untuk keberhasilan *framework*, memungkinkan generator membangun fondasi struktur visual yang kuat sebelum dioptimalkan untuk keterbacaan tekstual. Rincian protokol *curriculum learning 3-phase* (epoch ranges, bobot progresif, dan rencana validasi empiris) diuraikan pada Subbab IV.4.1 sebagai bagian dari strategi training.

Fungsi Loss Gabungan dan Konfigurasi Bobot Kelima komponen *loss* tersebut digabungkan menjadi satu fungsi *loss* total untuk Generator dengan bobot yang dikalibrasi:

$$L_G = \lambda_{adv}L_{adv} + \lambda_{L1}L_{L1} + \lambda_{perc}L_{perc} + \lambda_{rec-feat}L_{rec-feat} + \lambda_{CTC}L_{CTC} \quad (7)$$

Bobot optimal untuk setiap komponen *loss* (λ_{adv} , λ_{L1} , λ_{perc} , λ_{CTC}) dikalibrasi melalui Bayesian optimization pada Phase 3 *full training*. Konfigurasi lengkap dengan justifikasi setiap bobot, metodologi kalibrasi, dan analisis *magnitude scaling* diuraikan pada Tabel 7 (Subbab IV.4.2).

Konfigurasi Produksi Final Konfigurasi optimal yang digunakan dalam implementasi produksi (setelah ablasi sistematis):

$$L_G = 3.0 \cdot L_{adv} + 50.0 \cdot L_{L1} + 1.0 \cdot L_{perc} + 0.15 \cdot L_{CTC} \quad (8)$$

Konfigurasi **4-komponen** ini dirancang untuk mencapai keseimbangan *Pareto* optimal antara kualitas visual (PSNR, SSIM) dan keterbacaan teks (CER, WER):

- **Kualitas Visual:** Target PSNR >30 dB, SSIM ≥ 0.95 (hasil aktual: Bab V)
- **Keterbacaan HTR:** CER $34.9 \pm 1.2\%$, WER $47.2 \pm 1.8\%$
- **Efisiensi:** –8% waktu *training* vs konfigurasi 5-komponen
- **Stabilitas:** Konvergensi stabil tanpa *collapse* pada 100 *epochs*

Bobot λ_{L1} yang tinggi (50.0) memastikan kesetiaan piksel terhadap *ground truth*, menjaga detail struktural dokumen. Bobot CTC yang dikalibrasi dengan *curriculum learning* (0.05→0.10→0.15, rincian pada Subbab IV.4.1) mencegah *overfitting* pada aspek tekstual sambil tetap memberikan *guidance* keterbacaan yang kuat dan stabil.

IV.2 Lingkungan Eksperimen dan Spesifikasi Komputasi

Framework GAN-HTR yang dirancang pada subseksi sebelumnya diimplementasikan pada lingkungan komputasi *high-performance* dengan strategi *hardware* dan *software* yang dikalibrasi untuk *training* model generatif skala besar dengan memori tinggi. Bagian ini merinci konfigurasi lengkap infrastruktur eksperimen, mencakup spesifikasi *hardware*, *software stack*, strategi reproduktifitas, dan karakteristik performa aktual, untuk memastikan transparansi metodologis dan memenuhi kebutuhan non-fungsional NFR-4 yang telah ditetapkan pada Bab III.

Ringkasan Spesifikasi Kunci: Implementasi menggunakan NVIDIA RTX A4000 (16 GB VRAM), AMD Threadripper PRO 3955WX (16-core), 128 GB RAM, dengan TensorFlow 2.16.1/CUDA 12.8. *Training* produksi memerlukan 48.2 GPU-hours (50 epoch, batch size 2), mencapai *inference throughput* 17.2 citra/detik dengan latensi 30–35 ms per citra.

IV.2.1 Spesifikasi Perangkat Keras dan Justifikasi

Implementasi *framework* GAN-HTR dilakukan pada *workstation* dengan spesifikasi berikut:

Table 3: Spesifikasi lingkungan eksperimen

Komponen	Spesifikasi
GPU	2× NVIDIA RTX A4000 (16 GB GDDR6 per GPU)
CPU	AMD Ryzen Threadripper PRO 3955WX (16-core/32-thread)
RAM	128 GB DDR4-3200
Storage	1 TB NVMe SSD (Samsung 980 PRO)
OS	Ubuntu 22.04.5 LTS
Framework	TensorFlow 2.16.1 + CUDA 12.8 + cuDNN 8.9.7

Implementasi menggunakan RTX A4000 untuk stabilitas training jangka panjang dengan ECC memory, Threadripper PRO 3955WX dengan 16-core untuk data preprocessing paralel, dan 128 GB RAM untuk dataset caching.

Pemilihan Perangkat Keras GPU Implementasi menggunakan 2 unit NVIDIA RTX A4000 dengan 16 GB GDDR6 memory per GPU, dipilih untuk mendukung resolusi tinggi input (1024×128) dan ukuran model yang besar (89.4M parameter generator).

Alokasi GPU Training menggunakan satu GPU (GPU 0) via `CUDA_VISIBLE_DEVICES=0`. GPU kedua digunakan untuk eksperimen paralel dan *inference* simultan.

IV.2.2 Software Stack dan Konfigurasi Lingkungan

Konfigurasi *software* menggunakan Python 3.10, TensorFlow 2.16, CUDA 12.8, dan cuDNN 8.9. Semua dependensi di-*lock* melalui Poetry (`poetry.lock`) untuk reproduktifitas penuh.

Kebijakan Presisi Numerik Implementasi menggunakan Pure FP32 (*single precision*) untuk semua operasi *tensor*, tanpa *mixed precision* (FP16/FP32). Keputusan ini berdasarkan:

- *Stabilitas CTC Loss*: Kalkulasi CTC loss dengan *log-likelihood* memerlukan presisi penuh untuk menghindari *numerical underflow/overflow* pada sekuens panjang (hingga 128 karakter). Eksperimen preliminari dengan *mixed precision* mengalami NaN loss pada *epoch* 5–8.
- *Konsistensi Multi-Component Loss*: Lima komponen loss dengan *magnitude* berbeda ($L_{L1} \sim 0.3$, $L_{CTC} \sim 200$) memerlukan presisi konsisten untuk *gradient scaling* yang akurat.

- *Trade-off Performa*: FP32 mengorbankan $\sim 30\text{--}40\%$ kecepatan *training* dibanding *mixed precision*, tetapi memastikan konvergensi stabil tanpa *loss scaling* manual atau *gradient clipping* agresif.

Kebijakan ini dikonfigurasi secara eksplisit dalam skrip *training*:

```
tf.keras.mixed_precision.set_global_policy('float32')
```

Strategi Learning Rate Adaptif Implementasi menggunakan *learning rate schedule* adaptif untuk mencegah *overfitting* dan mempercepat konvergensi:

- *Warmup Phase* (10 epoch): *Learning rate* dinaikkan secara linear dari 10^{-5} ke nilai nominal (Generator: 2×10^{-4} , Discriminator: 2×10^{-4}) untuk stabilisasi awal *training*. Fase *warmup* mencegah gradien besar di *epoch* awal yang dapat menyebabkan *mode collapse*.
- *Annealing Phase* (20 epoch): Setelah *warmup*, *learning rate* diturunkan secara bertahap menggunakan *exponential decay* dengan *decay rate* 0.95 per 5 epoch.
- *ReduceLROnPlateau*: Jika metrik validasi tidak membaik selama 5 epoch berturut-turut, *learning rate* dikurangi 50% (*factor*=0.5) dengan batas minimum 10^{-6} . Strategi ini memastikan konvergensi halus mendekati optimum.

Konfigurasi *schedule* ini dikalibrasi berdasarkan eksperimen preliminari yang menunjukkan strategi *warmup* + *annealing* menghasilkan konvergensi $\sim 15\%$ lebih cepat dibanding *constant learning rate*, dengan stabilitas superior (tidak ada *loss spike* pada epoch 1–10).

Adaptive Loss Balancing Untuk menjaga keseimbangan optimal antara metrik visual (PSNR/SSIM) dan keterbacaan teks (CER/WER), implementasi menggunakan mekanisme *adaptive loss balancing* yang menyesuaikan bobot CTC *loss* secara dinamis berdasarkan rasio kontribusi komponen:

- *Target Rasio*: CTC *loss* target berkontribusi 40% dari total *loss*, komponen visual (L_1 + *perceptual* + *adversarial*) berkontribusi 60%, memastikan keseimbangan *Pareto*.
- *Penyesuaian Adaptif*: Setiap 100 *step*, bobot CTC (λ_{CTC}) disesuaikan berdasarkan rasio aktual:

$$\lambda_{CTC}^{(t+1)} = \lambda_{CTC}^{(t)} \times \left(1 + \alpha \times \left(\frac{r_{\text{target}} - r_{\text{actual}}}{r_{\text{target}}} \right) \right) \quad (9)$$

dengan $\alpha = 0.08$ adalah *adaptation rate*, $r_{\text{target}} = 0.40$, dan r_{actual} adalah rasio kontribusi CTC aktual.

- *Bounded Adjustment*: Bobot dibatasi dalam rentang $[0.05, 0.25]$ untuk mencegah dominasi berlebihan komponen tunggal.

Mekanisme adaptif ini dirancang untuk menjaga stabilitas *training* GAN-HTR dengan multi-komponen *loss* yang memiliki *magnitude* sangat berbeda ($L_{L1} \sim 0.3$

vs $L_{CTC} \sim 200$). Konfigurasi ini konsisten dengan seed management deterministik (seed=42) yang dijelaskan pada Subbab ?? untuk reproducibility.

Instalasi dan Konfigurasi *Environment* Implementasi menggunakan *virtual environment* Python yang dikelola oleh Poetry untuk isolasi *dependency*:

1. *Instalasi CUDA/cuDNN*: CUDA Toolkit 12.8 dan cuDNN 8.9.7 diinstal secara *native* pada sistem Ubuntu 22.04.5 LTS dengan NVIDIA Driver 545.23.08.
2. *Virtual Environment Poetry*: Poetry 1.7.1 membuat *virtual environment* terisolasi di `.venv/` dengan Python 3.10.12. File `poetry.lock` menjamin *exact dependency resolution* untuk reproduktifitas.
3. *Aktivasi Environment*: Setiap sesi *training/inference* dijalankan dalam *virtual environment* aktif:

```
poetry run python dual_modal_gan/scripts/train_enhanced.py
```

4. *Dependency Management*: `pyproject.toml` mendefinisikan *dependency* dengan *version constraints*, `poetry.lock` menyimpan *resolved versions* untuk *reproducible installs* lintas sistem.

IV.2.3 Reprodutifitas

Untuk memastikan reproduktifitas, kami mengkonfigurasi *environment variables* determinisme (Python hash seed, TensorFlow deterministic ops, cuDNN deterministic mode) dan alokasi memori dinamis GPU. Rincian konfigurasi lengkap disediakan di Appendix ?. Konfigurasi ini merupakan *trade-off* antara reproduktifitas (diperlukan validasi ilmiah) dan performa *training* (korban $\sim 5\text{--}10\%$ kecepatan).

Struktur Direktori dan Persistensi Data Implementasi menggunakan struktur direktori terorganisir untuk *data persistence* dan *artifact management*:

```
docRestoration/
|-- data/                                # Dataset TFRecord dan raw images
|   |-- tfrecords/                       # Training/validation/test splits
|   +-- synthetic_degraded/             # Generated degraded images
|-- models/                             # Model checkpoints
|   |-- recognizer_frozen/              # Pre-trained frozen recognizer
|   +-- checkpoints/                   # Training checkpoints (best models)
|-- mlruns/                             # MLflow experiment tracking
|   |-- 0/                             # Default experiment ID
|   +-- artifacts/                     # Model artifacts, configs, plots
|-- logs/                              # Training logs dan TensorBoard events
|   |-- training/                      # Stdout/stderr logs
|   +-- tensorboard/                   # TensorBoard event files
+-- outputs/                           # Inference results
```

```
|-- restored_images/      # Output restored images
+-- metrics/              # Evaluation metrics (CSV, JSON)
```

IV.2.4 Karakteristik Performa *Training* dan *Inference*

Pengukuran performa dilakukan pada konfigurasi produksi aktual untuk memberikan *baseline* reproduktifitas.

Table 4: Karakteristik performa pelatihan (50 *epoch*, *batch size* 2)

Metrik	Nilai
Total <i>Training Time</i>	48.2 GPU-hours (50 <i>epoch</i>)
Waktu per <i>Epoch</i>	~58 menit (<i>epoch</i> awal: 62 min, akhir: 54 min)
<i>Throughput Training</i>	~2.8 citra/detik (termasuk <i>backprop</i>)
VRAM Usage (<i>Training</i>)	14.4 GB / 16 GB (90% utilisasi)
VRAM Usage (<i>Validation</i>)	8.2 GB / 16 GB (<i>inference-only</i>)
CPU Usage	~35–45% (paralel <i>data loading</i>)
RAM Usage	~42 GB / 128 GB (<i>dataset caching</i>)
Disk I/O	~120 MB/s <i>read</i> (TFRecord <i>streaming</i>)
Power Consumption	~180W GPU + 95W CPU = 275W total

Metrik *Training* Catatan Konvergensi: *Training* mencapai konvergensi stabil pada *epoch* 44 (PSNR 30.74 dB, CER 34.9% pada *test set*; PSNR 30.91 dB, CER 27.1% pada *validation set*) dengan *early stopping patience* 25 *epoch* (konfigurasi konservatif untuk menghindari penghentian prematur). Total *checkpoint* tersimpan: 18 GB (3 *checkpoint* terbaik @ 6 GB per *checkpoint*).

Analisis Biaya Komputasi dan Energi: Dengan konsumsi daya rata-rata 275W selama 48.2 GPU-hours, total konsumsi energi adalah $275\text{W} \times 48.2\text{h} = 13.255\text{ kWh}$. Menggunakan tarif listrik Indonesia (~Rp 1.450/kWh atau \$0.10/kWh), biaya energi *training* adalah ~Rp 19.220 (\$1.32). Estimasi emisi karbon dengan *carbon intensity* grid Indonesia (~0.5 kg CO₂/kWh) adalah ~6.6 kg CO₂. Untuk konteks, ini setara dengan emisi berkendara mobil sejauh ~30 km. *Early stopping* dengan *patience* 25 *epoch* mencegah *training* berlebihan yang dapat menambah biaya komputasi tanpa peningkatan metrik signifikan.

Table 5: Karakteristik performa inferensi (citra baris 128 × 1024)

Metrik	Nilai
Latensi per Citra (<i>Single</i>)	50–60 ms (rata-rata: 54 ms)
<i>Throughput</i> (<i>Batch</i> =1)	~18.5 citra/detik
<i>Throughput</i> (<i>Batch</i> =2)	~17.2 citra/detik (30–35 ms per citra)
<i>Throughput</i> (<i>Batch</i> =4)	~15.8 citra/detik (25 ms per citra)
VRAM Usage (<i>Batch</i> =1)	6.2 GB / 16 GB
VRAM Usage (<i>Batch</i> =2)	7.8 GB / 16 GB
VRAM Usage (<i>Batch</i> =4)	10.4 GB / 16 GB
<i>Breakdown Waktu</i> (<i>Batch</i> =1)	<i>Preprocessing</i> : 8 ms, <i>Model</i> : 42 ms, <i>Postprocessing</i> : 4 ms

Metrik Inference Optimasi Inference: Untuk *deployment* produksi, *batch size* 2 memberikan *trade-off* optimal antara latensi (30–35 ms per citra) dan *throughput* (17.2 citra/detik), dengan VRAM *usage* moderat (7.8 GB) yang memungkinkan *concurrent inference* pada GPU yang sama.

Metodologi Validasi Performa Untuk memastikan metrik performa yang dilaporkan *robust* dan tidak dipengaruhi oleh *variance* eksperimental, validasi statistik dilakukan dengan protokol berikut:

1. **Coefficient of Variation (CV):** Digunakan untuk mengukur stabilitas metrik antar *runs*:

$$CV = \frac{\sigma}{\mu} \times 100\% \quad (10)$$

dengan σ adalah standar deviasi dan μ adalah rata-rata metrik. Kriteria penerimaan: $CV < 2\%$ untuk PSNR/SSIM, $CV < 5\%$ untuk CER/WER (metrik HTR lebih *noisy*).

2. **Independent Runs:** Tiga *independent runs* dengan *seed*=[42, 123, 456] dijalankan untuk validasi reproduktifitas. Hasil yang dilaporkan adalah rata-rata dari 3 *runs* dengan *confidence interval* 95%.
3. **Kriteria Akseptansi Hardware:** Validasi dilakukan pada 2 *workstation* terpisah dengan spesifikasi identik (RTX A4000, Threadripper PRO 3955WX) untuk memastikan hasil tidak spesifik terhadap konfigurasi *hardware* tunggal. Perbedaan metrik antar *workstation* harus $< 1.0\%$.
4. **Dokumentasi Variance:** Selain rata-rata, standar deviasi dan 95% *confidence interval* dilaporkan untuk metrik kritis (PSNR, SSIM, CER pada *test set*) di Bab V.

Protokol validasi ini memastikan metrik yang dilaporkan *statistically significant* dan dapat direproduksi oleh peneliti lain dengan konfigurasi serupa.

Reproducibility Checklist Untuk memastikan reproduktifitas penuh eksperimen, komponen berikut didokumentasikan dan di-*version control*:

1. **Poetry Lock File:** `poetry.lock` dengan *exact dependency versions* di-*commit* ke repositori Git (hash: a3f8b2e)
2. **Environment Variables:** Skrip *shell* (`scripts/setup_env.sh`) untuk konfigurasi konsisten *environment* sebelum *training*
3. **Random Seed Management:**
 - Python: `PYTHONHASHSEED=42, random.seed(42)`
 - NumPy: `numpy.random.seed(42)`
 - TensorFlow: `tf.random.set_seed(42)`
4. **Dataset Split Seed:** *Deterministic split* dengan *seed*=42 untuk pembagian *train/val/test* (70/15/15)
5. **Checkpoint Versioning:** `MLflow run_id` unik dengan *artifact logging* (model, *config*, metrik) untuk setiap eksperimen
6. **TensorFlow Determinism:** `TF_DETERMINISTIC_OPS=1, TF_CUDNN_DETERMINISTIC=1` dipaksa aktif

7. **cuDNN Benchmark Mode:** *Disabled* untuk determinisme (mengorbankan ~5–10% performa)
8. **Konfigurasi JSON:** File konfigurasi eksperimen (`configs/*.json`) di-*version control* dengan parameter lengkap

Dengan konfigurasi ini, eksperimen dapat direproduksi dengan *variance* $<0.5\%$ pada metrik PSNR/SSIM dan $<1.0\%$ pada CER/WER, divalidasi melalui 3 *independent runs* dengan *seed* identik pada sistem berbeda (verifikasi dilakukan pada 2 *workstation* terpisah dengan spesifikasi *hardware* sama).

Konfigurasi Monitoring dan Checkpoint Management Sistem monitoring terintegrasi untuk memastikan training berjalan stabil: (1) **Real-time tracking** menggunakan MLflow untuk logging loss components (adversarial, L1, perceptual, CTC) dan evaluation metrics (PSNR, SSIM, CER, WER) setiap epoch, dengan dashboard visualization untuk monitoring convergence, (2) **Checkpoint strategy** menyimpan model terbaik berdasarkan validation total generator loss dengan automatic cleanup checkpoint lama untuk efisiensi storage, naming convention `model_epoch{epoch}_valloss{loss:.4f}.h5`, (3) **Early stopping** dengan patience 25 epochs (telah dispesifikasi pada Tabel ??) untuk mencegah overfitting dengan automatic restoration ke best weights, dan (4) **Adversarial equilibrium monitoring** dengan target discriminator accuracy 60–80% untuk deteksi imbalance, menggunakan alert threshold jika accuracy $<55\%$ (generator terlalu kuat) atau $>85\%$ (discriminator terlalu dominan). Konfigurasi monitoring ini memastikan deteksi dini masalah training seperti mode collapse, gradient explosion, atau loss imbalance untuk intervensi manual jika diperlukan.

IV.3 Implementasi Pipeline Data dan Training Framework

Bagian ini mengoperasionalkan rancangan arsitektur dengan implementasi mencakup pipeline data preparation, strategi training, sistem monitoring, dan protokol evaluasi yang membentuk keseluruhan framework GAN-HTR.

Implementasi *pipeline* data merupakan langkah krusial dalam mewujudkan rancangan arsitektur GAN-HTR yang telah diuraikan sebelumnya. *Pipeline* ini mengintegrasikan kebutuhan fungsional terkait dataset sintetis dan dukungan untuk berbagai format *input*, seperti yang ditentukan dalam analisis kebutuhan. Dengan *pipeline* yang modular, sistem dapat menangani persiapan data untuk *training* Recognizer dan pembuatan dataset terdegradasi sintetis, memastikan kualitas data yang konsisten untuk eksperimen yang reproduktif.

IV.3.1 Pipeline Persiapan Dataset HTR

Fase pertama implementasi adalah persiapan dataset untuk training model Recognizer (arsitektur telah dijelaskan pada Subbab ??). *Pipeline* ini diimplementasikan dengan struktur modular:

Algorithm 1 Pipeline Persiapan Dataset HTR

- 1: **Input:** Halaman dokumen bersih, file XML transkripsi
 - 2: **Output:** TFRecord dataset (image, transcription)
 - 3: **1:** Segmentasi halaman menjadi baris teks menggunakan bounding box dari XML
 - 4: **2:** Ekstrak image patch untuk setiap baris teks
 - 5: **3:** Preprocessing: normalisasi intensitas, resize ke 128×1024
 - 6: **4:** Binarisasi: adaptive threshold untuk contrast enhancement
 - 7: **5:** Validasi: verifikasi kesesuaian image-text pairs
 - 8: **6:** Konversi ke format TFRecord untuk efisiensi I/O
 - 9: **7:** Split: pelatihan (80%), validasi (10%), pengujian (10%)
-

IV.3.2 Pipeline Degradasi Sintetis

Karena keterbatasan dataset dokumen terdegradasi untuk training (sesuai analisis kebutuhan Subbab ??), dikembangkan pipeline degradasi sintetis:

Algorithm 2 Pipeline Synthetic Degradation

- 1: **Input:** Dataset HTR bersih
 - 2: **Output:** Dataset triplet (degraded, clean, transcription)
 - 3: **1:** Load clean images dan transcriptions
 - 4: **2:** Apply degradations dengan probabilitas independen (dapat overlap):
 - 5: a. Bleed-through (30%): overlay mirrored text dengan opacity variabel
 - 6: b. Fading (40%): reduce intensity dengan exponential decay
 - 7: c. Stains (25%): add random brown spots dengan varying intensity
 - 8: d. Blur (20%): Gaussian filter dengan $\sigma \in [0.5, 2.0]$
 - 9: **3:** Add noise: salt & pepper (10% pixels) untuk simulasi sensor noise
 - 10: **4:** Validasi quality metrics (PSNR > 20 dB) untuk memastikan degradasi tidak ekstrem
 - 11: **5:** Simpan sebagai TFRecord triplet dengan struktur (degraded, clean, transcription)
-

Catatan: Probabilitas degradasi pada langkah 2 bersifat independen, sehingga setiap citra dapat mengalami kombinasi beberapa jenis degradasi secara simultan. Pendekatan ini mensimulasikan kondisi degradasi dokumen historis yang realistis, dimana dokumen umumnya mengalami multiple degradation types secara bersamaan.

IV.3.3 Format Data dan Preprocessing

Format TFRecord: Setiap sample disimpan dalam TFRecord dengan struktur:

- degraded_image: Tensor uint8 [128, 1024, 1]
- clean_image: Tensor uint8 [128, 1024, 1]
- transcription: String teks ground truth

- length: Int panjang sequence

IV.4 Strategi dan Protokol Training

Bagian ini menguraikan strategi training yang mengoperasionalkan rancangan arsitektur GAN adversarial dengan dual-modal discriminator, mencakup curriculum learning bertahap, loss balancing dinamis, dan kebijakan presisi numerik sebagai faktor kunci keberhasilan framework.

IV.4.1 Strategi Curriculum Learning dan Loss Balancing

Proses training mengadopsi strategi curriculum learning dengan tiga fase progresif untuk stabilitas konvergensi, di mana bobot CTC ditingkatkan secara bertahap (0.05→0.10→0.15). Strategi ini memungkinkan generator membangun fondasi struktur visual yang kuat sebelum dioptimalkan untuk keterbacaan tekstual:

Table 6: Protokol *Curriculum Learning* 3-Phase

Fase	Epoch	λ_{CTC}	Tujuan
Phase 1: Warmup	1–20	0.05	Stabilisasi struktur visual dasar
Phase 2: Ramp-up	21–40	0.10	Introduksi gradual <i>guidance</i> keterbacaan
Phase 3: Full Training	41–100	0.15	Optimasi penuh visual + keterbacaan

Justifikasi Parameter Curriculum Learning: Epoch ranges (Phase 1: 20, Phase 2: 20, Phase 3: 40-60) dan lambda values (0.05→0.10→0.15) ditentukan melalui eksperimen preliminary. Strategi ini mencegah dominasi awal CTC loss yang dapat merusak kualitas visual, memungkinkan generator membangun fondasi struktural sebelum fokus pada keterbacaan teks. Rincian justifikasi dan validasi empiris dilaporkan pada Bab V Subbab 5.2.

IV.4.2 Implementasi Multi-Component Loss Function

Fungsi loss generator dirancang untuk mengoptimalkan secara simultan kualitas visual dan keterbacaan tekstual melalui kombinasi terbobot dari lima komponen loss yang saling melengkapi. Setiap komponen menargetkan aspek spesifik dari restorasi dokumen HTR-oriented.

Total Loss Function: Loss function generator didefinisikan sebagai weighted sum dari lima komponen:

$$\mathcal{L}_G = \lambda_{adv}\mathcal{L}_{adv} + \lambda_{L1}\mathcal{L}_{L1} + \lambda_{perc}\mathcal{L}_{perc} + \lambda_{CTC}\mathcal{L}_{CTC} + \lambda_{rec-feat}\mathcal{L}_{rec-feat} \quad (11)$$

di mana λ adalah bobot yang dapat dikonfigurasi untuk menyeimbangkan kontribusi setiap komponen.

1. Adversarial Loss: Adversarial loss standar GAN mendorong generator menghasilkan citra yang realistis dan mampu menipu discriminator:

$$\mathcal{L}_{\text{adv}} = -\log D(I_{\text{gen}}, T_{\text{pred}}) \quad (12)$$

di mana D adalah discriminator dual-modal, $I_{\text{gen}} = G(I_{\text{deg}})$ adalah citra hasil restorasi, dan T_{pred} adalah representasi teks dari frozen recognizer. Loss ini memberikan sinyal untuk menghasilkan citra yang secara perseptual tidak dapat dibedakan dari ground truth.

2. Pixel Reconstruction Loss (L1): L1 loss memastikan kesamaan piksel-demi-piksel dengan ground truth:

$$\mathcal{L}_{L1} = \|I_{\text{gen}} - I_{\text{gt}}\|_1 = \sum_{i,j} |I_{\text{gen}}(i,j) - I_{\text{gt}}(i,j)| \quad (13)$$

Norma L1 dipilih karena menghasilkan hasil lebih tajam dibandingkan L2 dan lebih robust terhadap outliers. Loss ini memberikan sinyal rekonstruksi kuat untuk preservasi struktur dasar dokumen.

3. Perceptual Loss: Perceptual loss [?] membandingkan fitur tingkat tinggi dari jaringan VGG-19 praterlatih pada ImageNet:

$$\mathcal{L}_{\text{perc}} = \sum_{l \in L} \|\phi_l(I_{\text{gen}}) - \phi_l(I_{\text{gt}})\|_2 \quad (14)$$

di mana ϕ_l menunjukkan fitur dari lapisan l dari VGG-19. Fitur diekstrak dari multiple convolutional layers (conv1_2, conv2_2, conv3_4, conv4_4) untuk perbandingan perseptual multiskala, memastikan preservasi karakteristik visual dari detail tingkat rendah (edges, textures) hingga fitur semantik tingkat tinggi (pola goresan, bentuk karakter).

4. CTC Loss (HTR-Oriented): CTC loss dari frozen recognizer memberikan guidance eksplisit untuk keterbacaan teks:

$$\mathcal{L}_{\text{CTC}} = \text{CTC}(R(I_{\text{gen}}), Y_{\text{gt}}) \quad (15)$$

di mana R adalah frozen recognizer, Y_{gt} adalah ground truth transcription. CTC (Connectionist Temporal Classification) menghitung negative log-likelihood dari correct transcription given image.

Clipping Strategy untuk Stabilitas Numerik: CTC loss di-clip pada nilai maksimum 400.0 untuk mencegah gradient explosion ketika recognizer menghasilkan prediksi dengan confidence sangat rendah pada citra severely degraded:

$$\mathcal{L}_{\text{CTC}}^{\text{clipped}} = \min(\mathcal{L}_{\text{CTC}}, 400.0) \quad (16)$$

Threshold 400.0 dipilih berdasarkan analisis empiris distribusi CTC loss pada validation set (99th percentile), memastikan clipping hanya terjadi pada extreme outliers tanpa mempengaruhi mayoritas training samples. Strategi ini konsisten dengan precision policy (pure FP32) yang dijelaskan pada Subbab IV.4.4 untuk stabilitas numerik maksimal.

5. Recognition Feature Loss: Loss ini mendorong similarity pada feature space recognizer, providing complementary signal to CTC loss:

$$\mathcal{L}_{\text{rec-feat}} = \|\mathcal{F}_{\text{rec}}(I_{\text{gen}}) - \mathcal{F}_{\text{rec}}(I_{\text{gt}})\|_2^2 \quad (17)$$

di mana $\mathcal{F}_{\text{rec}}$ adalah feature extractor dari recognizer (output CNN backbone sebelum Transformer encoder). Loss ini ensures intermediate representations juga aligned, providing finer-grained guidance dibanding CTC loss yang hanya operates pada sequence level.

Catatan Konfigurasi Produksi: Pada konfigurasi optimal (Tabel 7), komponen ini menggunakan $\lambda_{\text{rec-feat}} = 8.0$ untuk eksperimen preliminary, namun pada konfigurasi produksi final (Subbab IV.4.1) diset $\lambda_{\text{rec-feat}} = 0.0$ (dinonaktifkan) setelah studi ablasi menunjukkan redundansi dengan CTC loss. Nilai 8.0 pada tabel merepresentasikan historical baseline dari fase eksperimen awal.

Konfigurasi Bobot Loss Optimal: Bobot loss ditentukan melalui grid search empiris pada validation set, balancing visual quality dan text readability:

Table 7: Konfigurasi bobot loss optimal: hasil grid search empiris untuk menyeimbangkan kualitas visual dengan keterbacaan teks

Komponen Loss	Bobot (λ)	Justifikasi
Adversarial ($\mathcal{L}_{\text{adv}}$)	3.0	Realisme visual, sinyal GAN moderate
Pixel/L1 ($\mathcal{L}_{\text{L1}}$)	50.0	Preservasi struktur dasar, sinyal kuat
Perceptual ($\mathcal{L}_{\text{perc}}$)	1.0	Preservasi topologi, baseline weight
CTC Loss ($\mathcal{L}_{\text{CTC}}$)	0.15	HTR guidance (scaled by curriculum)
Rec-Feature ($\mathcal{L}_{\text{rec-feat}}$)	8.0	Feature alignment, complementary to CTC

Pemilihan bobot didasarkan pada analisis trade-off multi-objective:

- **L1 Weight (50.0):** Bobot tertinggi untuk memastikan preservasi struktur dasar dokumen. Dominasi L1 mencegah over-smoothing yang sering terjadi dengan adversarial loss saja.
- **Rec-Feature Weight (8.0 $\rightarrow$ 0.0):** Nilai 8.0 pada tabel merepresentasikan konfigurasi preliminary experiments. Konfigurasi produksi final menggunakan 0.0 (dinonaktifkan) setelah ablation study menunjukkan redundansi dengan CTC loss ($\Delta\text{CER} < 0.5\%$, $p > 0.1$, computational cost +15%).

- **Adversarial Weight (3.0):** Bobot moderat untuk realisme tanpa instability. Adversarial loss memiliki variance tinggi, sehingga bobot terlalu besar dapat menyebabkan mode collapse.
- **CTC Weight (0.15):** Bobot rendah karena: (1) CTC loss memiliki magnitude lebih besar ($\sim 10-100$) dibanding losses lain ($\sim 0.1-10$), (2) Weight ini di-scale oleh curriculum learning phases ($0.05 \rightarrow 0.10 \rightarrow 0.15$, lihat Tabel 6), (3) Dengan scaling, kontribusi efektif CTC menjadi $\sim 12-15\%$ dari total loss pada Phase 3.
- **Perceptual Weight (1.0):** Baseline weight (normalized magnitude). Perceptual loss provides consistent gradient signal untuk preservasi semantic features.

Adaptive Loss Balancing (Implemented di IV.4): Untuk mencegah dominasi komponen loss tunggal, mekanisme adaptive balancing diimplementasikan dengan monitoring magnitude setiap komponen dan dynamic weight adjustment jika imbalance terdeteksi. Detail implementasi lengkap tersedia pada Subbab IV.4.2. Strategi ini bekerja secara sinergis dengan adaptive loss balancing yang dijelaskan pada Subbab IV.2 untuk memastikan keseimbangan optimal antara metrik visual dan keterbacaan tekstual.

Magnitude Scaling: Bobot CTC rendah (0.15 vs L1 50.0) didesain untuk mengkompensasi perbedaan magnitude natural. CTC loss tipikal $\sim 100-300$, L1 loss $\sim 0.1-0.5$, sehingga bobot dikalibrasi untuk kontribusi efektif yang sebanding.

IV.4.3 Implementasi Mode Diskriminator

Dual-modal discriminator mendukung dua mode operasi yang dapat dikonfigurasi sesuai dengan desain eksperimen yang telah diuraikan pada Bab III.4.3:

Ground Truth Mode: Mode ini menggunakan ground truth text sebagai input teks untuk diskriminator. Karakteristik:

- **Stabilitas:** Training lebih stabil karena teks konsisten dan bebas error
- **Kelebihan:** Konvergensi cepat, loss smooth, cocok untuk initial training
- **Kekurangan:** Kurang realistis untuk deployment karena tidak mensimulasikan kondisi inference
- **Use Case:** Ideal untuk warmup phase dan debugging architecture

Predicted Mode: Mode ini menggunakan predicted text dari frozen recognizer sebagai input. Karakteristik:

- **Realisme:** Mensimulasikan kondisi deployment dengan error recognition
- **Kelebihan:** Model belajar handle imperfect text, generalisasi lebih baik
- **Kekurangan:** Training lebih challenging, optimal untuk recognizer dengan CER $< 15\%$. Dengan recognizer CER 33.72% (Subbab ??), mode ini viable namun memerlukan training duration lebih panjang ($\sim 1.15x$ epochs) dan careful monitoring untuk convergence stability.

- **Use Case:** Ideal untuk fine-tuning dan evaluation yang realistis

Mode Selection untuk Implementasi Final: **Implementasi final menggunakan Ground Truth Mode** sebagai konfigurasi utama berdasarkan validasi preliminary experiments yang menunjukkan:

- **Stabilitas Superior:** Convergence 100% success rate vs 85% dengan Predicted Mode (5 independent runs)
- **Efisiensi Training:** Mencapai target PSNR 30 dB pada epoch 38 (avg) vs epoch 45 dengan Predicted Mode
- **Konsistensi Loss:** Standard deviation validation loss 35% lebih rendah, mengindikasikan training dynamics lebih predictable

Predicted Mode dieksplorasi sebagai ablation study (Bab V.4.3) untuk evaluasi robustness terhadap recognition errors. Hasil menunjukkan performa comparable (PSNR difference <0.3 dB, $p > 0.05$) namun dengan konvergensi 15% lebih lambat dan CER validation degradation +1.2%, mengonfirmasi Ground Truth Mode sebagai pilihan optimal untuk balance antara stabilitas dan performa.

Eksperimen Perbandingan: Perbandingan kuantitatif komprehensif kedua mode dilakukan melalui eksperimen terkontrol dengan metrik CER, WER, PSNR, dan SSIM. Hasil lengkap dan analisis statistical significance dilaporkan pada Bab V.4.3 (Ablasi Discriminator Mode).

IV.4.4 Implementasi Kebijakan Presisi

Kebijakan presisi pure FP32 diimplementasikan secara eksplisit untuk memastikan stabilitas numerik dalam training, terutama untuk kalkulasi CTC loss yang memerlukan presisi penuh.

Alasan Kebijakan FP32: Pemilihan pure FP32 (tanpa mixed precision FP16) didasarkan pada analisis stabilitas numerik:

1. **CTC Loss Numerical Precision:** Algoritma CTC melakukan komputasi log-probability pada sequence alignment yang sensitif terhadap underflow. Eksperimen preliminary dengan mixed precision FP16 (tested pada 3 runs) menunjukkan gradient underflow pada epoch 15–20, menyebabkan training divergence pada 1 dari 3 runs (33% failure rate).
2. **Multi-Component Loss Consistency:** Dengan 5 komponen loss berbeda (adversarial, L1, perceptual, CTC, rec-feat), presisi konsisten diperlukan untuk balancing yang stabil. FP16 dapat menyebabkan numerical inconsistency antar komponen karena perbedaan magnitude (adversarial loss ~ 0.1 – 1.0 , L1 loss ~ 10 – 50).
3. **Gradient Flow Stability:** Pure FP32 memastikan backpropagation melalui frozen recognizer (27.86M parameters) tidak mengalami precision loss yang dapat corrupt generator gradients.

Kebijakan pure FP32 dipilih untuk stabilitas numerik CTC loss dan reproducibility,

dengan trade-off training time $\sim 28\%$ lebih lambat dibanding mixed precision FP16. Validasi empiris cost-benefit analysis dilaporkan pada Bab V Subbab 5.2 (Ablasi Presisi Numerik).

Strategi training yang telah diuraikan pada subsection IV.4.1–IV.4.4 (curriculum learning, multi-component loss, discriminator mode, precision policy) membentuk metodologi inti framework GAN-HTR. Implementasi strategi ini memerlukan konfigurasi discriminator dan sistem evaluasi yang tepat untuk memastikan adversarial equilibrium dan monitoring performa secara menyeluruh, yang akan diuraikan pada bagian berikutnya.

Catatan: Pipeline pelatihan Recognizer yang digunakan sebagai frozen component (metodologi, rationale, curriculum learning strategy) telah diuraikan pada Bab III Fase 2. Subsection IV.4 fokus pada strategi training framework GAN-HTR yang mengintegrasikan recognizer tersebut sebagai guidance mechanism.

IV.5 Protokol Evaluasi dan Validasi Eksperimen

Setelah arsitektur dirancang, environment dikonfigurasi, pipeline data diimplementasikan, dan strategi training ditetapkan, tahap berikutnya adalah merancang protokol evaluasi eksperimental untuk memvalidasi efektivitas framework yang diusulkan.

Protokol evaluasi dan validasi eksperimen dirancang untuk memvalidasi efektivitas framework GAN-HTR yang diusulkan melalui eksperimen komparatif terkontrol dan studi ablasi sistematis. Protokol ini mengimplementasikan metodologi yang telah dijelaskan pada Bab III.5 (Metodologi Evaluasi) dengan fokus pada: (1) perbandingan dengan baseline methods untuk mengukur peningkatan performa, (2) ablasi sistematis komponen untuk isolasi kontribusi individual, (3) validasi statistical significance, dan (4) reproducibility protocol untuk memastikan hasil dapat diverifikasi independen. Detail metodologi metrik (PSNR, SSIM, CER, WER) dan statistical testing (paired t-test, Bonferroni correction) telah diuraikan pada Bab III.5 dan tidak diulang di sini.

IV.5.1 Baseline Methods untuk Komparasi Kuantitatif

Framework yang diusulkan dievaluasi melalui perbandingan kuantitatif dengan metode baseline yang dapat direplikasi pada kondisi eksperimental identik (dataset, recognizer, hardware). Baseline dipilih untuk isolasi kontribusi spesifik: (1) classical methods (tanpa deep learning) untuk mengukur gain dari GAN approach, (2) GAN single-modal (tanpa dual-modal discriminator) untuk mengukur kontribusi arsitektur dual-modal.

Table 8: Metode baseline untuk eksperimen komparatif

Kategori	Metode	Deskripsi
Baseline Klasik (Tanpa Deep Learning)		
B1	No Enhancement	HTR langsung pada citra terdegradasi
B2	Otsu Thresholding	Binarisasi global berbasis histogram
B3	Sauvola Adaptive	Binarisasi adaptif lokal untuk non-uniform lighting
Baseline GAN Internal		
B4	GAN Single-Modal	Generator + Discriminator CNN visual-only (tanpa dual-modal, tanpa CTC loss)

Variabel Terkontrol: Semua metode dievaluasi pada test set identik ($n=712$), menggunakan frozen recognizer yang sama untuk CER/WER, dengan parameter baseline klasik (B2, B3) dioptimalkan via grid search pada validation set. Metrik evaluasi: PSNR, SSIM (visual quality) dan CER, WER (text readability).

Hypotheses: (H1) Framework proposed menghasilkan CER/WER signifikan lebih rendah vs baseline klasik ($p < 0.05$), (H2) Dual-modal discriminator (proposed) menghasilkan CER/WER lebih rendah vs GAN single-modal ($p < 0.05$).

IV.5.2 Studi Ablasi Komponen Loss Function

Studi ablasi sistematis dirancang untuk isolasi kontribusi individual setiap komponen loss function melalui pendekatan inkremental (menambahkan satu komponen per konfigurasi). Konfigurasi ablasi mengikuti protokol yang telah dirancang pada Bab III.2.3.4 dengan 5 eksperimen: (Exp 01) Pixel loss only (U-Net baseline), (Exp 02) +Adversarial (GAN standard), (Exp 03) +Perceptual (VGG-based), (Exp 04) +CTC (HTR-aware, **konfigurasi optimal**), (Exp 05) +Recognition feature (validasi redundansi). Detail rationale setiap konfigurasi telah dijelaskan pada Bab III dan tidak diulang di sini untuk menghindari redundansi.

Alasan Exp 04 sebagai Optimal: Konfigurasi 4-komponen (pixel + adversarial + perceptual + CTC) dipilih sebagai *production configuration* berdasarkan preliminary experiments yang menunjukkan Exp 05 (+recognition feature) tidak memberikan gain signifikan ($\Delta\text{CER} < 0.5\%$, $p > 0.1$) namun menambah computational cost +15%. Validasi empiris lengkap dilaporkan pada Bab V dengan actual performance metrics dan statistical tests.

Statistical Testing: Paired t-test untuk CER/WER comparison antar konfigurasi ($\alpha = 0.05$), Bonferroni correction untuk multiple comparisons ($\alpha = 0.0125$ untuk 4 comparisons), Cohen's d untuk effect size ($d \geq 0.5$ medium, $d \geq 0.8$ large). Kriteria signifikansi: $p < 0.0125$ AND $d \geq 0.5$. Detail metodologi statistical testing telah dijelaskan pada Bab III.5.

IV.5.3 Ablasi Arsitektur Discriminator

Untuk isolasi kontribusi dual-modal discriminator, eksperimen terkontrol dengan generator identik (U-Net Enhanced V2) namun discriminator berbeda: (Config A) CNN single-modal visual-only, (Config B) CNN+BiLSTM dual-modal visual+textual. Controlled variables: generator, loss function

(Pixel+Adv+Percep+CTC), training hyperparameters, dan dataset identik. **Hypothesis H3:** Dual-modal (Config B) menghasilkan CER/WER signifikan lebih rendah vs single-modal (Config A) dengan $p < 0.05$ dan Cohen's $d \geq 0.5$.

IV.5.4 Protokol Validasi Set Uji

Evaluasi final pada test set independen ($n=712$, tidak pernah dilihat selama training/validation) dengan protokol: (1) Model selection dari Exp 04 (optimal config) berdasarkan validation loss, (2) Evaluasi jalan tunggal (tanpa pemilihan selektif) untuk integritas hasil, (3) Comprehensive metrics (PSNR, SSIM, CER, WER, inference time), (4) Analisis kualitatif pada 20 sampel representatif (degradasi ringan/sedang/berat). Target performa: PSNR >30 dB, SSIM ≥ 0.90 , CER reduction $\geq 25\%$ dari baseline terdegradasi. Hasil evaluasi actual dilaporkan pada Bab V.

IV.5.5 Protokol Reprodutibilitas

Untuk memastikan hasil eksperimen dapat diverifikasi secara independen, protokol reproducibility mencakup: (1) **Random seed management** dengan seed deterministik (seed=42) untuk Python, NumPy, TensorFlow, dan dataset split, (2) **Independent runs** dengan minimal 3 runs menggunakan seeds berbeda (42, 123, 456) untuk validasi variance, hasil dilaporkan sebagai mean $\pm 95\%$ confidence interval, (3) **Variance criteria** dengan coefficient of variation (CV) $<2\%$ untuk PSNR/SSIM dan CV $<5\%$ untuk CER/WER sebagai kriteria akseptansi reproducibility, (4) **Hardware validation** pada 2 workstations terpisah dengan spesifikasi identik (RTX A4000, Threadripper PRO 3955WX) untuk memastikan hasil tidak hardware-specific (acceptable difference $<1.0\%$), dan (5) **Artifact versioning** dengan MLflow run_id untuk tracking model checkpoints, configurations, dan training curves. Konfigurasi determinism (TF_DETERMINISTIC_OPS=1, cuDNN deterministic mode) dan dependency versioning (Poetry lock file) telah dijelaskan pada Subbab ???. Protokol ini memungkinkan peneliti lain mereproduksi eksperimen dengan variance $<0.5\%$ pada visual metrics dan $<1.0\%$ pada HTR metrics.